🚀 Collections Framework Masterclass

Chapter 17 — TreeMap — Complete Deep Dive

> **Interview Importance:** ⭐⭐⭐⭐⭐

`TreeMap` is the primary Java `Map` implementation when you need **sorted keys**.

The central idea:

> **TreeMap stores mappings in a balanced Red-Black Tree, ordered by key.**

Unlike `HashMap`, where ordering isn't guaranteed, `TreeMap` continuously maintains its keys according to their natural ordering or a supplied `Comparator`.

---

1. What is TreeMap?

Package:

```
java.util.TreeMap
```

Hierarchy:

```
Map
 ↓
SortedMap
 ↓
NavigableMap
 ↓
TreeMap
```

Conceptually:

```
                TreeMap
                   ↓
            Red-Black Tree
                   ↓
             Sorted Keys
```

2. Basic Example

```
Map<Integer, String> map =
    new TreeMap<>();

map.put(30, "C");
map.put(10, "A");
map.put(20, "B");
```

Iteration:

```
10 → A
20 → B
30 → C
```

The insertion order doesn't matter.

The keys are maintained in sorted order.

---

3. TreeMap Is Sorted by Key

This is critical:

```
TreeMap<K, V>
```

sorts according to:

```
K → Key
```

not:

```
V → Value
```

Example:

```
TreeMap<Integer, String> map =
    new TreeMap<>();

map.put(30, "Apple");
map.put(10, "Zebra");
map.put(20, "Mango");
```

Order:

```
10 → Zebra
20 → Mango
30 → Apple
```

The values aren't sorted.

---

4. Why Use TreeMap?

Use TreeMap when you need:

- Sorted keys
- Range queries
- First/last key
- Nearest key
- Floor/ceiling operations
- Navigational operations

For example:

> "Find the greatest key less than or equal to X."

That's exactly the kind of problem TreeMap handles efficiently.

---

5. Internal Data Structure

TreeMap uses a:

> **Red-Black Tree**

Conceptually:

```
            20
           /
         10    30
        / \    /
       5  15  25 35
```

The actual implementation stores entries as tree nodes.

Conceptually each node contains:

```
key
value
left
right
parent
color
```

---

6. What Is a Red-Black Tree?

A Red-Black Tree is a **self-balancing Binary Search Tree**.

It maintains additional rules involving node colors:

```
RED
BLACK
```

These rules ensure the tree doesn't become excessively skewed.

Therefore its height remains:

```
O(log n)
```

which gives TreeMap predictable logarithmic operations.

---

7. Why Not a Normal Binary Search Tree?

Consider inserting:

```
10
20
30
40
50
```

A normal BST could become:

```
10

   20

      30

         40

            50
```

Now the tree behaves almost like a linked list.

Search becomes:

```
O(n)
```

A Red-Black Tree rebalances itself:

```
       20
      /
    10    40
         /
       30  50
```

Height stays approximately logarithmic.

---

8. Red-Black Tree Properties

The classic rules are:

Rule 1

Every node is either:

```
RED
```

or:

```
BLACK
```

Rule 2

The root is black.

Rule 3

Null leaves are treated as black.

Rule 4

A red node cannot have a red child.

```
RED
 ↓
RED
```

is invalid.

Rule 5

Every path from a node to its descendant null leaves contains the same number of black nodes.

These rules maintain balance.

---

9. Do You Need to Memorize Every Red-Black Rule?

For a Java interview:

Know the concept and why it is used.

You usually don't need to implement a complete Red-Black Tree unless the interviewer specifically asks.

The key answer:

> TreeMap uses a Red-Black Tree to maintain sorted keys while keeping operations approximately O(log n).

---

10. TreeMap `put()`

When:

```
map.put(key, value);
```

is called:

```
put()
 ↓
Compare key
 ↓
Move left or right
 ↓
Find insertion position
 ↓
Insert node
 ↓
Rebalance tree
```

The comparison determines where the key belongs.

---

11. Natural Ordering

If no Comparator is supplied, keys must generally implement:

```
Comparable
```

Example:

```
TreeMap<Integer, String> map =
    new TreeMap<>();
```

`Integer` implements `Comparable<Integer>`.

Therefore TreeMap knows:

```
10 < 20 < 30
```

---

12. Comparable vs Comparator

This distinction is essential.

Comparable

Defines natural ordering **inside the class**.

```
class Employee
        implements Comparable<Employee> {

    @Override
    public int compareTo(Employee other) {
        return this.id - other.id;
    }
}
```

Usage:

```
TreeMap<Employee, String> map =
    new TreeMap<>();
```

Comparator

Defines ordering **outside the class**.

```
TreeMap<Employee, String> map =
    new TreeMap<>(
        Comparator.comparingInt(
            Employee::getSalary
        )
    );
```

Now the TreeMap sorts keys by salary.

---

13. Comparator Takes Priority

If you supply:

```
new TreeMap<>(comparator)
```

the TreeMap uses that Comparator for key ordering rather than relying on the key's natural ordering.

This is useful when:

```
Same class
+
Different sorting requirements
```

are needed.

---

14. Custom Comparator Example

Suppose:

```
class Employee {
    String name;
    int salary;
}
```

Sort keys by salary:

```
TreeMap<Employee, String> map =
    new TreeMap<>(
        Comparator.comparingInt(
            e -> e.salary
        )
    );
```

Now TreeMap's ordering is determined by salary.

---

15. Critical Comparator Trap

TreeMap uses its ordering comparison to determine key equivalence.

Suppose:

```
Comparator<Employee> comparator =
    Comparator.comparingInt(
        Employee::getSalary
    );
```

Two different Employees with the same salary can compare as:

```
compare(a, b) == 0
```

TreeMap can therefore treat them as the **same key for Map purposes**, even if:

```
a.equals(b) == false
```

This is a major interview concept.

---

16. `compareTo() == 0` and Key Uniqueness

For TreeMap, key uniqueness is effectively determined by:

```
compareTo() / Comparator
```

not simply by `equals()`.

Example:

```
Employee A → salary 50000
Employee B → salary 50000
```

If comparator only compares salary:

```
compare(A, B) == 0
```

TreeMap treats them as equivalent keys.

Therefore:

> **TreeMap's ordering should generally be consistent with equals when possible.**

---

17. `get()` Internals

For:

```
map.get(key);
```

TreeMap starts at the root:

```
Root
 ↓
compare()
 ↓
key smaller?
 ↙       ↘
left     right
 ↓         ↓
compare again
```

It continues until:

```
Found
```

or:

```
Not found
```

Because the tree is balanced:

```
O(log n)
```

18. TreeMap Complexity

Operation	Complexity
`put()`	O(log n)
`get()`	O(log n)
`remove()`	O(log n)
`containsKey()`	O(log n)
`firstKey()`	O(log n) worst-case
`lastKey()`	O(log n) worst-case
`higherKey()`	O(log n)
`lowerKey()`	O(log n)
`ceilingKey()`	O(log n)
`floorKey()`	O(log n)

TreeMap trades HashMap's average O(1) lookup for:

```
Guaranteed sorted structure
+
Navigational operations
```

---

19. `firstKey()`

Returns the smallest key.

```
map.firstKey();
```

Example:

```
10
20
30
```

returns:

```
10
```

---

20. `lastKey()`

Returns the largest key.

```
map.lastKey();
```

returns:

```
30
```

---

21. `firstEntry()` and `lastEntry()`

You can retrieve the complete mapping:

```
map.firstEntry();
map.lastEntry();
```

Example:

```
firstEntry()
→ 10 = A

lastEntry()
→ 30 = C
```

---

22. `lowerKey()`

Returns the greatest key **strictly less than** the specified key.

Suppose:

```
10
20
30
40
```

Then:

```
map.lowerKey(30);
```

returns:

```
20
```

---

23. `floorKey()`

Returns the greatest key **less than or equal to** the specified key.

```
map.floorKey(30);
```

returns:

```
30
```

If:

```
map.floorKey(35);
```

returns:

```
30
```

---

24. `higherKey()`

Returns the smallest key **strictly greater than** the specified key.

```
map.higherKey(30);
```

returns:

```
40
```

---

25. `ceilingKey()`

Returns the smallest key **greater than or equal to** the specified key.

```
map.ceilingKey(30);
```

returns:

```
30
```

For:

```
map.ceilingKey(35);
```

returns:

```
40
```

---

26. The Four Important Navigation Methods

Memorize this table:

Method	Meaning
`lowerKey(k)`	`< k`
`floorKey(k)`	`<= k`
`ceilingKey(k)`	`>= k`

Method	Meaning
`higherKey(k)`	`> k`

Visual:

```
10   20   30   40
          ↑
          k
```

Think:

```
lower    → left, strictly
floor    → left or same
ceiling  → right or same
higher   → right, strictly
```

---

27. Entry Versions

Every major navigation operation has an Entry equivalent:

```
lowerEntry()
floorEntry()
ceilingEntry()
higherEntry()
```

Instead of returning only the key, they return:

```
Map.Entry<K,V>
```

Example:

```
Map.Entry<Integer, String> entry =
    map.floorEntry(35);
```

---

28. `pollFirstEntry()`

Returns and removes the smallest mapping.

```
map.pollFirstEntry();
```

Example:

```
10 → A
20 → B
30 → C
```

After:

```
map.pollFirstEntry();
```

result:

```
20 → B
30 → C
```

---

29. `pollLastEntry()`

Returns and removes the largest mapping.

```
map.pollLastEntry();
```

Useful for:

- Min/max processing
- Priority-style workflows
- Ordered data management

---

30. Descending Map

TreeMap provides:

```
map.descendingMap();
```

This gives a reverse-order view.

Normal:

```
10
20
30
```

Descending:

```
30
20
10
```

---

31. `descendingKeySet()`

Returns keys in descending order:

```
map.descendingKeySet();
```

Useful when you need reverse traversal without copying the Map.

---

32. Range Queries

One of TreeMap's biggest advantages is range querying.

Suppose:

```
10
20
30
40
50
```

You want:

```
20 through 40
```

Use:

```
map.subMap(20, true, 40, true);
```

Result:

```
20
30
40
```

---

33. `subMap()`

Modern form:

```
subMap(
    fromKey,
    fromInclusive,
    toKey,
    toInclusive
)
```

Example:

```
map.subMap(20, true, 40, false);
```

Result:

```
20
30
```

Because:

```
20 included
40 excluded
```

34. `headMap()`

Returns entries before a specified key.

```
map.headMap(30);
```

Typically:

```
10
20
```

There is also:

```
map.headMap(30, true);
```

which includes `30`.

---

35. `tailMap()`

Returns entries from a specified key onward.

```
map.tailMap(30);
```

Typically:

```
30
40
50
```

There is also:

```
map.tailMap(30, false);
```

which excludes `30`.

---

36. Range Views Are Views

This is important.

Methods such as:

```
subMap()
headMap()
tailMap()
```

return **views**, not necessarily independent copies.

Therefore modifications can affect the original TreeMap.

Conceptually:



This is useful but requires care.

---

37. NavigableMap

`TreeMap` implements:

```
NavigableMap<K,V>
```

which provides navigation operations such as:

```
lower
floor
ceiling
higher
```

and:

```
pollFirstEntry()
pollLastEntry()
descendingMap()
```

So the hierarchy is:

```
Map
 ↓
SortedMap
 ↓
NavigableMap
 ↓
TreeMap
```

---

38. SortedMap vs NavigableMap

SortedMap

Provides:

```
firstKey()
lastKey()
subMap()
headMap()
tailMap()
```

NavigableMap

Adds:

```
lower
floor
ceiling
higher
```

plus descending and polling operations.

Therefore:

`NavigableMap` is an enhanced version of `SortedMap`.

---

39. Null Keys

This is a common interview trap.

TreeMap generally does **not** permit a null key when using natural ordering.

```
TreeMap<Integer, String> map =
    new TreeMap<>();

map.put(null, "A");
```

This results in a `NullPointerException` under natural ordering.

Why?

Because TreeMap needs to compare keys.

---

40. Null Values

Null values are generally allowed:

```
map.put(10, null);
```

The issue is the **key comparison**, not the value.

---

41. Can a Comparator Support Null Keys?

Potentially yes, if the supplied Comparator explicitly handles null.

For example:

```
TreeMap<Integer, String> map =
    new TreeMap<>(
        Comparator.nullsFirst(
            Comparator.naturalOrder()
```

```
        )
    );
```

Now the comparator defines how null should be ordered.

The important point is:

> Null-key behavior depends on the ordering mechanism being used.

---

42. TreeMap Is Not Thread-Safe

Like HashMap and LinkedHashMap:

```
TreeMap
```

is not inherently thread-safe.

For concurrent sorted-map requirements, Java provides:

```
ConcurrentSkipListMap
```

rather than a synchronized TreeMap implementation.

We'll cover concurrent collections separately.

---

43. TreeMap vs HashMap

Feature	HashMap	TreeMap
Internal structure	Hash table	Red-Black Tree
Ordering	None guaranteed	Sorted keys
Average `get()`	O(1)	O(log n)
`put()`	O(1) avg	O(log n)
Range queries	Not natural	Excellent
Navigation	❌	✅
Null key	One allowed	Generally no

Feature	HashMap	TreeMap
Thread-safe	❌	❌

Use:

```
HashMap
→ fastest general-purpose lookup
```

Use:

```
TreeMap
→ sorted/navigable key operations
```

---

44. TreeMap vs LinkedHashMap

Feature	LinkedHashMap	TreeMap
Ordering	Insertion/access	Sorted
Structure	Hash table + links	Red-Black Tree
Lookup	O(1) avg	O(log n)
Range operations	Limited	Excellent
Navigation	❌	✅
LRU use	✅	❌
Null key	One allowed	Generally no

---

45. When TreeMap Is the Right Choice

Use TreeMap when the requirement sounds like:

> "Keep keys sorted."

or:

> "Find the nearest key."

or:

> "Find all entries between X and Y."

or:

> "Find the smallest key greater than X."

These are strong TreeMap signals.

---

46. Real-World Example — Price Ranges

Suppose:

```
Price → Product
```

and you need:

> Find the highest price not exceeding ₹5000.

With TreeMap:

```
Integer price =
    products.floorKey(5000);
```

This is much cleaner than scanning every entry.

---

47. Real-World Example — Scheduling

Suppose:

```
timestamp → task
```

You need:

> Find the next scheduled task after time T.

Use:

```
map.higherEntry(timestamp);
```

This is a natural TreeMap use case.

---

48. Real-World Example — Range Query

Suppose:

```
Employee ID → Employee
```

You need:

> Employees with IDs between 1000 and 2000.

A TreeMap can provide a range view:

```
map.subMap(
    1000,
    true,
    2000,
    true
);
```

---

49. `Comparator` Consistency Trap

Consider:

```
Comparator<String> comparator =
    (a, b) ->
        Integer.compare(
            a.length(),
            b.length()
        );
```

Then:

```
TreeMap<String, Integer> map =
    new TreeMap<>(comparator);

map.put("Java", 1);
map.put("Code", 2);
```

Both strings have length 4.

Therefore:

```
compare("Java", "Code") == 0
```

The second insertion can replace the first mapping.

The Map effectively considers them equivalent under its ordering.

This is why Comparator design matters enormously with TreeMap.

---

50. Mutable Keys and TreeMap

HashMap has a mutable-key problem involving:

```
hashCode()
```

TreeMap has a related problem involving:

```
ordering
```

If the fields used by the Comparator/`compareTo()` change after insertion, the tree's ordering assumptions can become invalid.

Example:

```
Employee salary = 50000
        ↓
insert into TreeMap
        ↓
salary changes to 90000
```

If salary determines ordering, the node's physical location may no longer correspond to its new comparison result.

Therefore:

> Don't mutate fields that determine key ordering while the key is stored in a TreeMap.

---

51. TreeMap `put()` Mental Model

```
put(key, value)
       ↓
Compare with root
       ↓
  ┌────┴────┐
  ↓         ↓
smaller    larger
  ↓         ↓
left       right
  ↓         ↓
repeat comparison
       ↓
insert node
       ↓
rebalance
```

The Red-Black Tree maintains balance after insertion.

---

52. TreeMap `remove()` Mental Model

```
remove(key)
      ↓
Search tree
      ↓
Find node
      ↓
Remove node
      ↓
Rebalance
      ↓
Tree remains balanced
```

Therefore:

```
O(log n)
```

---

53. Tree Rotations

Rebalancing can use:

```
Left Rotation
Right Rotation
```

Conceptually:

```
Right rotation:

        30
       /
     20
    /
   10
```

becomes approximately:

```
      20
     /
   10    30
```

Rotations change structure while preserving sorted-order relationships.

---

54. Why Rotation Works

Before:

```
30
/
20
```

```
/
10
```

After:

```
20
/
10 30
```

The in-order ordering remains:

```
10 < 20 < 30
```

but the tree becomes much more balanced.

Red-Black Tree rules determine when rotations/recoloring are necessary.

---

55. Does TreeMap Store Sorted Values?

No.

It stores:

```
Sorted keys
```

The values remain associated with their keys but don't influence tree ordering unless the values are actually part of the key/comparator design.

---

56. Can TreeMap Have Duplicate Values?

Yes.

```
map.put(10, "A");
map.put(20, "A");
```

is perfectly valid.

---

57. Can TreeMap Have Duplicate Keys?

No.

But remember:

> "Duplicate" is determined by the TreeMap's comparison.

If:

```
compare(k1, k2) == 0
```

TreeMap treats them as equivalent keys for Map purposes.

---

58. `firstKey()` vs `firstEntry()`

```
firstKey()
```

returns:

```
K
```

while:

```
firstEntry()
```

returns:

```
Map.Entry<K,V>
```

Likewise:

```
lastKey()
lastEntry()
```

---

59. `pollFirstEntry()` vs `firstEntry()`

This distinction is easy:

```
firstEntry()
→ inspect only

pollFirstEntry()
→ inspect + remove
```

Similarly:

```
lastEntry()
→ inspect

pollLastEntry()
→ inspect + remove
```

---

60. ⭐ Interview Scenario

Requirement

Need to maintain:

```
Student ID → Student
```

in sorted ID order.

Use:

```
TreeMap<Integer, Student>
```

---

61. ⭐ Interview Scenario

Requirement

Need the nearest key ≤ X.

Use:

```
map.floorKey(x);
```

This is a strong TreeMap use case.

---

62. ⭐ Interview Scenario

Requirement

Need the nearest key ≥ X.

Use:

```
map.ceilingKey(x);
```

---

63. ⭐ Interview Scenario

Requirement

Need all mappings in a range.

Use:

```
map.subMap(
    from,
    true,
    to,
    true
);
```

---

64. ⭐ Interview Scenario

Requirement

Need the next key after X.

Use:

```
map.higherKey(x);
```

---

65. ⭐ Interview Scenario

Requirement

Need the previous key before X.

Use:

```
map.lowerKey(x);
```

---

66. ⭐ Coding Question

What is the output?

```
TreeMap<Integer, String> map =
    new TreeMap<>();

map.put(30, "C");
map.put(10, "A");
map.put(20, "B");

System.out.println(map.firstKey());
System.out.println(map.lastKey());
```

Output:

```
10
30
```

---

67. ⭐ Coding Question

Given:

```
10, 20, 30, 40
```

What does:

```
map.floorKey(25);
```

return?

```
20
```

What does:

```
map.ceilingKey(25);
```

return?

```
30
```

---

68. ⭐ Coding Question

Given:

```
10, 20, 30, 40
```

```
map.lowerKey(30);
```

returns:

```
20
```

while:

```
map.higherKey(30);
```

returns:

```
40
```

Remember:

```
lower  → <
higher → >
floor  → <=
ceiling → >=
```

---

69. ⭐ Tricky Question

Does TreeMap use `equals()` to determine whether two keys are the same?

Not directly.

TreeMap relies on its ordering:

```
compareTo()
```

or:

```
Comparator.compare()
```

If comparison returns zero, the keys are considered equivalent for Map operations.

---

70. ⭐ Tricky Question

Why can two objects that are not equal according to `equals()` still behave as the same TreeMap key?

Because the Comparator may return:

```
compare(a, b) == 0
```

TreeMap uses that ordering comparison to identify equivalent keys.

---

71. ⭐ Tricky Question

Why is TreeMap slower than HashMap for simple lookup?

Because:

```
HashMap
→ hash calculation
→ bucket
→ O(1) average
```

while:

```
TreeMap
→ tree traversal
→ O(log n)
```

But TreeMap provides capabilities HashMap doesn't:

```
sorting
+
range queries
+
navigation
```

---

72. TreeMap Decision Framework

```
Need Map
   ↓
Need sorted keys?
   ↓
  YES
```

```
   ↓
TreeMap
```

Need:

```
floor
ceiling
lower
higher
range
```

→ **TreeMap**

Need only:

```
fast average lookup
```

→ **HashMap**

Need:

```
insertion/access order
```

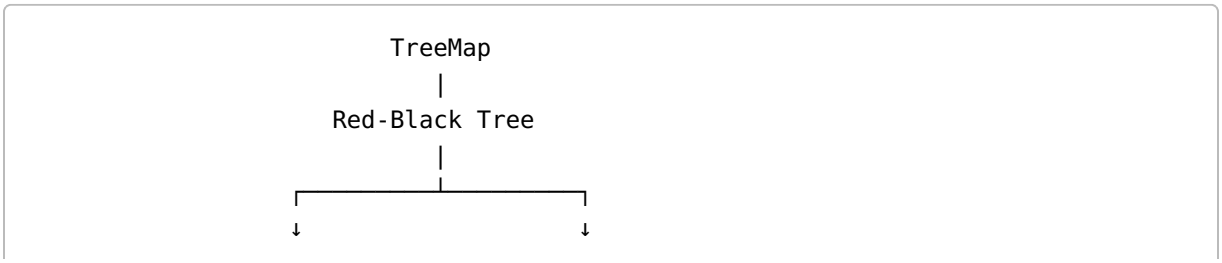
→ **LinkedHashMap**

Need:

```
concurrent sorted map
```

→ **ConcurrentSkipListMap**

🧠 Final Mental Model

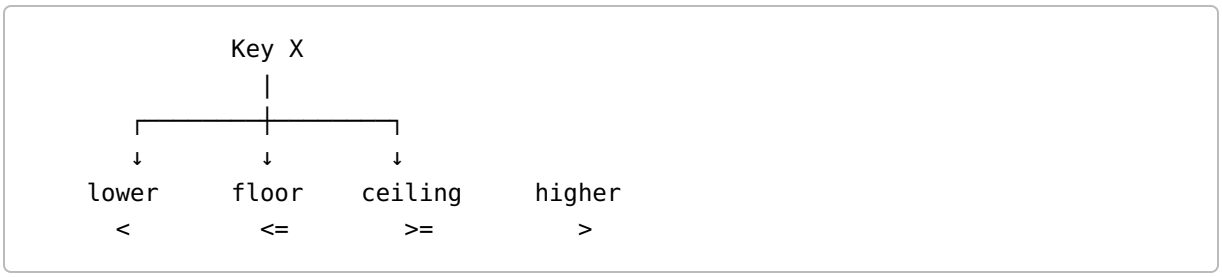


```
 Left subtree       Right subtree
smaller keys        larger keys
```

Every key follows:

```
Key
 ↓
Comparator / compareTo()
 ↓
left or right
 ↓
balanced tree
```

Navigation:



And the main trade-off:

```
HashMap
→ O(1) average lookup
→ no ordering

TreeMap
→ O(log n)
→ sorted + navigable
```

🔥 Chapter Summary

Concept	Key Point
TreeMap	Sorted Map
Internal structure	Red-Black Tree
Ordering	Key ordering

Concept	Key Point
Natural ordering	`Comparable`
Custom ordering	`Comparator`
`put()`	O(log n)
`get()`	O(log n)
`remove()`	O(log n)
`firstKey()`	Smallest key
`lastKey()`	Largest key
`lowerKey()`	`< key`
`floorKey()`	`<= key`
`ceilingKey()`	`>= key`
`higherKey()`	`> key`
`subMap()`	Range view
`headMap()`	Keys before boundary
`tailMap()`	Keys from boundary
`descendingMap()`	Reverse-order view
Null key	Generally not with natural ordering
Null values	Allowed
Duplicate keys	No
Duplicate values	Yes
Thread-safe	No
Concurrent sorted alternative	`ConcurrentSkipListMap`

🎯 Interview Readiness Checklist

You should be able to explain:

- [ ] What TreeMap is
- [ ] Why TreeMap uses a Red-Black Tree
- [ ] Why Red-Black Tree is preferred over an unbalanced BST
- [ ] Red-Black Tree balancing concept

- [ ] TreeMap `put()` flow
- [ ] TreeMap `get()` flow
- [ ] TreeMap `remove()` flow
- [ ] `Comparable` vs `Comparator`
- [ ] Why Comparator consistency with equals matters
- [ ] Why `compare() == 0` can mean equivalent TreeMap keys
- [ ] `firstKey()` / `lastKey()`
- [ ] `lowerKey()` / `floorKey()`
- [ ] `ceilingKey()` / `higherKey()`
- [ ] `subMap()` / `headMap()` / `tailMap()`
- [ ] Why range results are views
- [ ] `descendingMap()`
- [ ] TreeMap vs HashMap
- [ ] TreeMap vs LinkedHashMap
- [ ] Null-key behavior
- [ ] Mutable ordering-key problem
- [ ] Why TreeMap isn't thread-safe
- [ ] When to choose TreeMap

🚀 Next Chapter

Chapter 18 — WeakHashMap (Complete Deep Dive)

We'll focus only on what's new:

- Strong vs weak references
- How WeakHashMap interacts with Garbage Collection
- WeakReference and ReferenceQueue
- Why entries can disappear automatically
- Why `size()` can change unexpectedly
- WeakHashMap vs HashMap
- WeakHashMap vs caching
- Real-world use cases
- Common GC-related interview traps
- 20+ interview questions